Review: Arrays and Linked Lists

CSI2110 – Data Structures and Algorithms

Lecture study notes

Source document: `../0/Review.pdf`. These notes synthesize the existing review and clarify boundary cases. References use PDF page numbers.

Contents

1 Array memory model **2**

2 Singly linked lists **2**

2.1 Tail-pointer question 2

3 Doubly linked lists **2**

4 Comparison **3**

5 Self-check **3**

1 Array memory model

Source: PDF pages 1. The review illustrates contiguous Java integer slots. An `int` is a 32-bit value; the teaching model places consecutive slots four bytes apart:

Index	Illustrative address	Value
0	1000	10
1	1004	20
2	1008	30
3	1012	40

If the base is b and slot width w, slot i is modeled at $b + wi$. This explains constant-time indexed access. Actual Java addresses/layout are managed by the JVM and not exposed as these raw addresses. Object arrays hold references, not all referenced objects contiguously.

2 Singly linked lists

Source: PDF pages 1–2. Each node stores an element and a next reference. Nodes may occupy unrelated addresses, such as 1000 → 5000 → 2400 → `null`. Access follows references, rather than adding a byte offset.

The source insertion example assumes a positive index and a valid predecessor. A complete version handles the head and validates indices:

```
insertAt(i, value):
    require 0 <= i <= size
    if i == 0:
        head = new Node(value, head)
        if size == 0: tail = head
    else:
        pred = head
        repeat i - 1 times: pred = pred.next
        p = new Node(value, pred.next)
        pred.next = p
        if i == size: tail = p
    size = size + 1
```

The pointer change is $O(1)$; locating the predecessor costs $O(i)$, so indexed insertion is $O(n)$ worst case. Save the old successor before changing the predecessor's next reference.

2.1 Tail-pointer question

A tail reference gives $O(1)$ append but does not identify the tail's predecessor. Removing the last node is therefore still $O(n)$ in a singly linked list. Head removal is $O(1)$.

3 Doubly linked lists

Source: PDF pages 2. Each node also stores a previous reference. A known interior node x can be removed by rewiring `x.prev.next = x.next` and `x.next.prev = x.prev`.

The review guards null neighbors; a complete list must also update head/tail for endpoint removal, otherwise stale references can remain:

```
removeKnownNode(x):
    if x.prev != null: x.prev.next = x.next
    else: head = x.next
    if x.next != null: x.next.prev = x.prev
    else: tail = x.prev
    size = size - 1
    invalidate x
```

This takes $O(1)$ given a valid node in the list. Index-based removal adds traversal and remains $O(n)$ worst case. Sentinels simplify boundaries, but are not user entries.

4 Comparison

Operation	Array sequence	Singly linked*	Doubly linked*
Read index	$O(1)$	$O(n)$	$O(n)$
Local insertion	$O(n)$ worst	$O(1)$ with predecessor	$O(1)$ with position
Local removal	$O(n)$ worst	$O(1)$ with predecessor	$O(1)$ with position
Append	$O(1)$ if room	$O(1)$	$O(1)$
Remove last	$O(1)$	$O(n)$	$O(1)$

*Assume maintained head and tail references. Array growth may copy $O(n)$ entries; doubling gives amortized $O(1)$ append.

Key idea. Separate the time to locate an entry from the time to update its neighbors. Constant-time linked updates assume the needed references are available.

5 Self-check

1. Draw the references after inserting 15 between 10 and 20.
2. Removing the only node must leave both head and tail null.
3. Explain why a singly linked queue is efficient but a singly linked deque is not: the queue removes at the head, while a deque must also remove at the tail.